Material-by-Material Audit & Technical Decision

1. Comer Textbook: *Internetworking With TCP/IP Vol III* (Ch. 1–12)

- **What is inside:** Client-server rendezvous problem, server privilege and security (Authentication, Authorization, Data Security, Privacy, Protection), connection-oriented (TCP) vs. connectionless (UDP), stateless vs. stateful servers, idempotent operations, iterative vs. concurrent servers, master/slave processes, and server deadlock. [1]
- **Decision for your project: Core Foundation (Use Heavily).**
    - **Security Checklist:** Your Spring Boot backend acts as the privileged server and must implement the exact five duties listed in Ch. 2.3.2: **Authentication** (login/JWT), **Authorization** (separating React staff from Kotlin clients), **Data Security** (HTTPS), **Privacy**, and **Protection** against resource abuse.
    - **Stateless & Idempotent APIs:** Ch. 2.3.6–2.3.8 warns that stateful servers fail when clients crash or networks duplicate packets. Design your Spring Boot REST endpoints to be **stateless** and **idempotent** (using a unique loan request ID) so retried mobile requests do not create duplicate loans.
    - **Server Deadlock (Ch. 8.26):** Avoid blocking I/O patterns where the server waits indefinitely for a slow mobile client while holding database connections. [1]

2. `SWE4107`: *WWW and Client-Server Model* (Lecture Slides)

- **What is inside:** Two-tier vs. three-tier architectures, Fat Client vs. Fat Server (Host-based, Server-based, Cooperative, Client-based processing), client-side validation (JavaScript/HTML) vs. server-side processing, and dynamic content generation from databases. [1]
- **Decision for your project: Direct Architectural Match.**
    - **Cooperative / Three-Tier Processing:** Split your application logic cleanly. Use **React and Kotlin** (Presentation + light Application logic) for form input validation before sending data over the network. Keep **Spring Boot** (Application + Database logic) responsible for loan eligibility calculations and **Neon PostgreSQL** (DBMS) for storage. [1]

3. `SWE4107` Chapter 2: *Introduction to Internet and WWW* ()

- **What is inside:** History of the Internet/ARPANET, ICANN, IETF, W3C, Internet backbones, basic protocols (HTTP, FTP, SMTP, POP3), and web browsers. [1]
- **Decision for your project: Background Only (Minimal Implementation Impact).**
    - Only the **HTTP request/response** model applies directly to how React and Kotlin communicate with your Spring Boot server. [1]

4. *Concurrent Processing in Client-Server Software* (Comer Ch. 3, `CS423-cotter` Slides & Dr. Dakshayini Notes)

- **What is inside:** Apparent vs. real concurrency, timeslicing, context switching overhead, local vs. global variables in concurrent threads, Thread-on-Demand

vs. **Thread Pool**, Socket pre-allocation, and **Delayed Concurrency** [cite: 6]. [1]

- **Decision for your project: Critical for Backend Performance.**
  - **Thread Pool Model:** Instead of spawning a new OS process (`fork()`) per loan request, Spring Boot's embedded Tomcat uses a fixed **Thread Pool** (from the `CS423` slides) to assign worker threads to incoming HTTP requests and return them to the pool afterward.
  - **Local vs. Global Variables:** As warned in Ch. 3.4.2, concurrent threads share global memory. Never store a borrower's loan details in a class-level field inside a Spring `@Service` or `@Controller`; keep request data strictly inside local method variables.
  - **Delayed Concurrency:** Handle fast read queries (checking loan status) immediately on the request thread, while offloading slow tasks (credit scoring or PDF generation) to background worker threads. [1]

5. *Sockets Programming in C* & *Socket API Notes* (`COMP2330` Slides & Shadrack Notes)

- **What is inside:** BSD Sockets vs. WinSock, Stream Sockets (`SOCK_STREAM` / TCP) vs. Datagram Sockets (`SOCK_DGRAM` / UDP), socket lifecycle (`socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`, `close`), and Network Byte Order (`htons`/`htonl`) [cite: 5]. [1]
- **Decision for your project: Handled Under the Hood (Abstraction Layer).**
  - You will **not** write raw C `socket()` or `htons()` calls. However, you are applying the **Stream Socket (TCP)** model because banking requires guaranteed, ordered, error-free byte streams (HTTP over TCP) rather than unreliable UDP datagrams [cite: 5]. [1]

6. *Remote Procedure Call (RPC)* (22-Slide Deck & Text Notes)

- **What is inside:** Client/server stubs, parameter marshalling/unmarshalling, Interface Definition Languages (IDL), External Data Representation (XDR) for big-endian/little-endian compatibility, Dynamic Binding via a directory/binder, and RPC failure modes (server crash, client crash/orphans, lost replies) [cite: 4]. [1]
- **Decision for your project: Conceptual Equivalent (Replaced by REST/JSON).**
  - Instead of C-based DCE RPC and XDR, your system uses **REST over HTTP**. Your **OpenAPI/Swagger** specification acts as the **IDL**, **Jackson** (in Spring Boot) and **Gson/Kotlinx.serialization** (in Kotlin) act as the **marshalling stubs** converting objects into machine-independent **JSON** (replacing XDR), and **DNS/URL routing** replaces the DCE binder [cite: 4]. [1]

7. *Distributed Objects & Java RMI* (`BIT2216` Decks)

- **What is inside:** Remote interfaces (`java.rmi.Remote`), Stub/Skeleton layers, Remote Reference Layer, RMI Registry (port 1099), Distributed Garbage Collection (reference counting), and **Invocation Semantics** (**Maybe**, **At-least-once**, **At-most-once** via request retransmission, duplicate filtering, and reply caching) [cite: 3]. [1]
- **Decision for your project: Adapt Invocation Semantics (Skip Raw Java RMI).**

    - Do **not** use Java RMI because your clients are **React (JavaScript)** and **Kotlin (Android)**, and your slide notes explicitly state *"in RMI, all objects must be written in Java"*.
    - **Use At-Most-Once Invocation Semantics:** Apply the fault-tolerance table from `BIT2216` Slide 42 (Retransmit request + **Duplicate filtering** + Retransmit cached reply) to your Spring Boot loan submission endpoint so a loan is processed **at most once** even if the mobile app retries on a flaky network. [1]

8. *Communications Middleware* & `CS6407` *Middleware* Decks

- **What is inside:** Middleware architecture (hiding heterogeneity between OS and applications), synchronous Request/Reply (RPC/OOM) vs. asynchronous **Message-Oriented Middleware (MOM)** (persistent queues, `MQput`/`MQget`, JMS) vs. **Event-Based Middleware (Publish/Subscribe)** (topic-based and content-based pub/sub, Composite Event Detection), and **Web Services** (SOAP, WSDL, UDDI). [1]
- **Decision for your project: Hybrid Pattern (Request/Reply + Pub/Sub).**
    - **Synchronous Request/Reply:** Use REST/JSON (the modern lightweight evolution of Web Services) for immediate actions like logging in, submitting a loan form, or viewing loan balances.
    - **Event-Based Middleware (Pub/Sub):** Use Spring's `ApplicationEventPublisher` (or a message broker) to decouple side effects. When a loan officer approves a loan in React, publish a `LoanApprovedEvent`; subscribers (Notification Service, Audit Logger) react asynchronously without blocking the officer's screen. [1]

9. Classroom Photo: *Real-Time Transaction Systems* (`image_vddKv_.png`)

- **What is inside:**
    1. Correctness requires **both database consistency and meeting deadlines** (**Hard**, **Firm**, and **Soft** deadlines).
    2. Disk I/O has **wide variance in execution times** compared to main memory.
    3. **Waits for locks, transaction aborts, and resource contention** remain major problems even with fast memory.
    4. System design must ensure **enough processing power to meet deadlines without excessive hardware**.
- **Decision for your project: Crucial Constraint for Your Database & Backend Design.**
    - **Deadlines:**
        - *Firm Deadline (zero value after deadline):* OTP verification codes and loan offer acceptance timers must strictly expire in PostgreSQL/Spring Boot; late submissions are rejected.
        - *Soft Deadline (diminishing value if late):* Automated credit eligibility checks and dashboard queries must return quickly so users do not abandon the app.
    - **Reducing Disk Variance & Lock Contention (PostgreSQL on Neon):**
        - Because Neon is a cloud database over the network, disk/network latency varies. Use **HikariCP connection pooling** (pre-allocated connections) and proper **database indexes** to keep working data in PostgreSQL's shared memory buffers.

- **Avoid Lock Waits & Aborts:** If two loan officers open and approve the same loan simultaneously, long database locks cause waits or deadlocks. Use **Optimistic Concurrency Control** (a `version` integer column in PostgreSQL) so conflicting updates fail fast and cleanly without holding locks.

Detailed Project Prompt (Grounded Strictly in Your Course Materials)

Prompt:
Act as a Distributed Systems and Full-Stack Software Engineer. Design and implement an **Online Bank Loan Processing System** that directly applies the Client-Server Computing, Middleware, Concurrency, and Real-Time Transaction concepts from our course syllabus.

Technology Stack:

- **Client UI (Borrowers):** Kotlin (Android)
- **Staff UI (Loan Officers):** React (Web)
- **Application Server:** Java Spring Boot (REST API)
- **Database:** PostgreSQL (hosted on Neon)

Technical Requirements Mapped to Course Concepts:

1. **Three-Tier Cooperative Architecture (`SWE4107`):**
   - Implement client-side input validation in React and Kotlin to minimize unnecessary network round-trips.
   - Keep all sensitive loan scoring, interest calculation, and approval logic on the Spring Boot server.
2. **Server Security Responsibilities (Comer Ch. 2.3.2):**
   - Implement **Authentication** (JWT tokens), **Authorization** (role separation between `CLIENT` and `OFFICER`), **Data Security** (HTTPS/TLS), **Privacy** (hiding internal borrower fields via DTOs), and **Protection** (request rate/payload limits).
3. **Stateless Server & At-Most-Once Invocation Semantics (Comer Ch. 2.3.6–2.3.8 & `BIT2216`):**
   - Keep the Spring Boot server completely stateless.
   - Implement **At-Most-Once semantics** (duplicate filtering + cached reply return) using an `Idempotency-Key` table in PostgreSQL so retried mobile requests from Kotlin never trigger duplicate loan applications or disbursements.
4. **Server Concurrency & Thread Safety (Comer Ch. 3 & `CS423`):**
   - Configure Tomcat's **Thread Pool** and HikariCP's connection pool (socket pre-allocation).
   - Ensure all Spring `@Service` classes use only local method variables (no shared mutable instance state).

    - Implement **Delayed Concurrency** by executing slow tasks (credit report checks, audit logging) asynchronously off the main HTTP thread.
5. **Middleware Communication Patterns (`CS6407`):**
    - Use synchronous Request/Reply (REST + JSON marshalling via DTOs) for client-server queries.
    - Use Event-Based **Publish/Subscribe** middleware (Spring Application Events) to decouple loan status changes (`LOAN_APPROVED`, `LOAN_REJECTED`) from downstream notification and logging subscribers.
6. **Real-Time Transaction Constraints (Lecture Slide):**
    - Enforce **Firm Deadlines** on loan offer acceptance windows and verification codes (rejecting requests past `expires_at`).
    - Minimize **lock contention and transaction aborts** between concurrent loan officers using **Optimistic Locking** (`@Version` column in PostgreSQL) instead of pessimistic database locks.
    - Optimize PostgreSQL indexes to reduce disk I/O variance and meet **Soft Deadlines** for loan status queries.

Output Requested:

1. **System Architecture & Concept Mapping Table** linking each component to the course principles above.
2. **PostgreSQL DDL Schema** (`users`, `loans`, `repayments`, `idempotency_records`, `audit_events`) incorporating `version` columns and deadline timestamps.
3. **Java Spring Boot Implementation** (Models, Repositories, Idempotent & Optimistically-Locked Loan Service, Pub/Sub Event Listeners, and REST Controllers).
4. **Client Integration Snippets** for Kotlin (Retrofit with Idempotency header) and React (Loan Officer approval dashboard handling version conflicts).